STADIO

SCHOOL OF INFORMATION TECHNOLOGY

Assignment Code	SS3
Assignment Type	Individual Project
NQF Level	6
Total Marks	120

Module Name	Web Systems and Technologies
Module Code	WST262

- Consult Canvas for the due dates, the Assessment Guidelines for Students document, the Assignment Overview, and other updated details. Check the Announcements by your lecturer and use the Module Q&A forum to ask questions related to this assessment.
- For rules and regulations applicable to assessments, refer to the Student Handbook.
- Note STADIO's rules relating to plagiarism and cheating. Do not hesitate to seek help if you are unsure of how to reference your sources. Make use of the STADIO Referencing Guide
- Submit the assessment as one PDF document on Canvas. The document must have a cover page which includes your details and module details.
- **If you have not submitted their SS1 or SS2, they will be allowed to do so along with their SS3, however their SS1 and SS2 will only be given a maximum of 50% of the grade.**

INTRODUCTION

Scenario: Ward 42 Fix-It, a community service-reporting web app

The Mamelodi East Residents' Association is a volunteer organisation that works with the ward councillor's office in Ward 42, Tshwane. Every week residents report potholes, burst water pipes, broken streetlights and illegal dumping. At present these reports arrive as WhatsApp voice notes, photos and phone calls to one volunteer coordinator, Mr Tshepo Mahlangu. Reports get lost, residents do not know whether anything is being done, and the councillor's office cannot see which problems are most common.

The association has received a small community grant and has asked you, a junior web developer at Mzansi Digital Labs (a small software company in Pretoria), to build **Ward 42 Fix-It**: a web application that residents can use on their phones to log a fault with a photo, and that the coordinator can use to manage and close reports.

What the association needs

- **Residents** can submit a report with a category, a street name, a short description and an optional photo, and can then look up the status of their report by its reference number.

- **The coordinator** can log in, see all reports in a table, filter them by category and status, update a report's status (Open, In progress, Closed), and view a simple dashboard that counts reports by category and status.
- **A short tutorial video** on the home page shows residents how to submit a report.
- **Mobile and low-data use:** most residents use entry-level Android phones on prepaid mobile data, often with weak signal. The app must be responsive, light on data, and installable as a Progressive Web App (PWA) that still shows a useful page when the phone is offline.
- **Accessibility:** many residents are older people, and some use screen readers or enlarge the text on their phones. The app must follow the POUR principles and WCAG guidance in Topic 6 of the Study Guide.
- **Security and privacy:** reports contain residents' names, cell phone numbers and photos of their streets. The application must protect this personal information, in line with the Protection of Personal Information Act 4 of 2013 (POPIA), and must resist the vulnerabilities covered in Topic 7 of the Study Guide.

Table 1 lists the minimum data the system must store for each report.

Field	Type	Notes
reference	Text	Unique, generated by the system, for example W42-0153
category	Text	Pothole, Water leak, Streetlight, Illegal dumping or Other
street	Text	Street name and nearest house number or landmark
description	Text	Required; 10 to 500 characters
reporter_name	Text	Required; personal information
reporter_cell	Text	Required; South African cell number of 10 digits starting with 0; personal information
photo_path	Text	Optional; path to the stored image file, not the image itself
status	Text	Open, In progress or Closed; new reports are Open
created_at	Date and time	Set by the server when the report is saved

Table 1: Minimum data for each report

OVERVIEW

In this project you will analyse, build, secure and deploy Ward 42 Fix-It. The project covers all five module outcomes and ends with a reflective question on your own development process. It consists of seven parts and counts **120 marks** in total.

Part	Title / Description	Module outcome	Marks
A	Web system architecture and HTTP analysis	MO1	15
B	Responsive web application with a framework	MO2	30
C	Mobile delivery as a Progressive Web App	MO3	15
D	Security analysis and correction	MO4	22
E	Digital media storage, transfer and retrieval	MO5	18
F	Reflective question	All	10

Part	Title / Description	Module outcome	Marks
G	Deployment, documentation and professional presentation	All	10
	TOTAL		**120**

Module outcomes assessed

- **MO1:** Demonstrate an understanding of the major components of a web system and how they function together, including the web server, database, analytics, and front end.
- **MO2:** Design and implement a responsive web application utilising a web framework and presentation technologies.
- **MO3:** Develop a mobile app that is usable, efficient, and secure on more than one device.
- **MO4:** Analyse a web or mobile system and correct security vulnerabilities.
- **MO5:** Implement storage, transfer, and retrieval of digital media in a web application with appropriate file, database, or streaming formats.

Please take note of the below AI traffic light matrix.

ARTIFICIAL INTELLIGENCE TRAFFIC LIGHT MATRIX (AI TLM)				
Section	**RED**	**AMBER**	**GREEN**	**Detail of AI use**
	May NOT use AI	**May use AI**	**MUST use AI**	
Parts A, B, C, D, E and G		✓		You MAY use a generative AI tool to explain a concept, an error message or a section of documentation (for example the Flask or Bootstrap documentation), and to check the clarity of your report. Provide proof of AI usage as part of your submission. Note the below: • You may NOT use AI to generate any HTML, CSS, JavaScript or Python code, your architecture diagram, or your answers to the security analysis in Question D1. • You may NOT use AI to generate any text that is used in the report. • You may NOT rely on AI for statements of South African law (including POPIA) or for published standards such as WCAG; verify these against the primary source and cite accordingly. • You MUST include a short AI Usage Declaration (half a page maximum) stating which tool you used, what you asked it, and at least one instance where you rejected or corrected its output, with your reason.
Part F	✓			You may NOT use a generative AI tool at any point in the preparation of Part F, including for structuring, prompting, editing or proofreading. *Part F is an account of your own reasoning about your own work on this project. No AI system has access to what you thought, and text*

				about your reasoning that was not written by you is a false statement about yourself rather than an unattributed source. • AI-generated or AI-edited text detected in Part F will be treated as academic misconduct and referred in terms of the STADIO Plagiarism Policy, not marked down. • Your AI Usage Declaration must state explicitly that no AI tool was used in Part F.

NOTE:

- STADIO employs AI writing detection tools to monitor AI use.
- Presenting work produced by AI as your own contravenes the STADIO Plagiarism Policy.
- Use of paraphrasing tools to circumvent the above specifications is considered an immediate forfeiture of all marks.
- Refer to the STADIO Referencing Guide for guidance. All work must be referenced.

INSTRUCTIONS

Read the scenario in the Introduction and then complete Parts A to G that follow.

- This is an individual project. All design work, code and evidence must be your own.
- Build the application with **Flask** (back end), **Bootstrap 5** (responsive layout), **HTML5, CSS and JavaScript** (front end) and **SQLite** (database), using **VS Code** as set up in Section 1.5 of the Study Guide. You may use Flask-WTF and Flask-SQLAlchemy.
- Use **fictitious** names, cell phone numbers and photos in all test data and screenshots. Do not use real residents' personal information.
- Submit **one PDF report** that contains a cover page, your answers to every question in order, and the screenshots and code extracts requested. Code extracts must use a monospace font such as Consolas.
- Also submit your complete project folder in **one ZIP file** named `Surname_StudentNumber_WST262.zip`, and give the link to your **GitHub repository** on the cover page of your report (Part G). Do not include the venv folder or the database file in the ZIP or the repository.
- Your application must run with `flask run` after `pip install -r requirements.txt`. An application that does not run is marked on its code alone and cannot earn the marks for working features.
- Reference the Study Guide, the official documentation you consult (Flask, Bootstrap, MDN, WCAG) and any other sources according to the STADIO Referencing Guide.

PART A: WEB SYSTEM ARCHITECTURE AND HTTP ANALYSIS (15 MARKS)

Before you build anything, show how the parts of Ward 42 Fix-It will work together (Study Guide, Topics 1 and 3).

Question A1: Architecture diagram and request flow (8 marks)

- Draw an architecture diagram of Ward 42 Fix-It that shows the front end (browser and PWA), the Flask back end, the API endpoints, the SQLite database, the file storage for photos, and the analytics dashboard.

Label every connection with the protocol or data format it uses (for example HTTPS, JSON or multipart/form-data).

- Using the five-step interaction, explain step by step what happens when a resident submits a report with a photo, from the moment they tap Submit until the confirmation page appears.
- Justify in a short paragraph why a relational (SQL) database suits this data better than a NoSQL database, referring to Table 1.

Question A2: HTTP analysis **(7 marks)**

Run your finished application locally. Use Wireshark or any tool of your choice to capture:

- a GET request for the home page; and
- the POST request that submits a new report with a photo.

For each request, include a screenshot and a table that identifies the method, URL, status code, the Content-Type of the request and the response, and two other headers. Explain the difference between the two requests, and why the report form must be served over HTTPS once it is deployed.

PART B: RESPONSIVE WEB APPLICATION WITH A FRAMEWORK (30 MARKS)

Build the Ward 42 Fix-It application. Your application must have at least these pages: **Home** (with the tutorial video), **Report a fault**, **Check status**, **Coordinator login**, **All reports** and **Dashboard**.

Question B1: Semantic, accessible and responsive front end **(10 marks)**

- Use a shared Jinja base template with semantic HTML5 elements (`header`, `nav`, `main`, `footer` and others as appropriate) and a correct heading hierarchy on every page.
- Use the Bootstrap grid and components so that every page works at phone (360 px), tablet (768 px) and desktop (1280 px) widths. Add at least one media query of your own in an external CSS file.
- Apply the POUR principles: labels on every form field, alt text on images, sufficient colour contrast, relative font sizes, and full keyboard operation.
- Include screenshots of three pages at all three widths.

Question B2: Flask back end, database and dashboard **(10 marks)**

- Create a database model that stores every field in Table 1, and generate the reference number on the server.
- Implement Flask routes to create a report, look up a report's status by reference number, list and filter all reports, and update a report's status.
- Protect the coordinator pages with a login. Store the coordinator's password as a hash, never as plain text.
- Build the Dashboard page, which shows the number of reports per category and per status. This is the analytics component of your system.

Question B3: JavaScript interactivity **(10 marks)**

- Client-side validation on the report form, using DOM events, that checks the description length and the cell number format, and shows clear error messages next to the fields without reloading the page.
- A live filter on the All-reports page that hides rows that do not match the selected category or status, without a new request to the server.

- A saved draft: if a resident loses signal while filling in the form, the text they have typed is kept in `localStorage` and restored when they return; the draft is cleared after a successful submission. Do not store the reporter's name or cell number in the draft, and explain why.
- Use an external JavaScript file and handle errors with `try/catch` where appropriate.

PART C: MOBILE DELIVERY AS A PROGRESSIVE WEB APP (PWA) (15 MARKS)

Question C1: Convert Fix-It into a PWA (8 marks)

- Add a web app manifest with the app's name, short name, icons, theme colour, start URL and display mode
- Register a service worker that caches the app shell (HTML, CSS, JavaScript and icons) and shows a custom offline page that tells the resident they are offline and that their draft is saved (Study Guide, Section 8.3.4).
- Show, with screenshots, that the app installs on a phone or from a desktop browser, and that the offline page appears when the network is switched off.

Question C2: Usability and efficiency on more than one device (7 marks)

- Test the app on at least two different devices (for example an Android phone and a laptop, or two phones with different screen sizes). Record each device, browser, screen width and any problem found, and how you fixed it.
- Run a Lighthouse audit (Chrome developer tools) on the Report a fault page in mobile mode and include the scores. Describe three changes you made, or would make, to reduce the amount of data the page uses on prepaid mobile data.

PART D: SECURITY ANALYSIS AND CORRECTION (22 MARKS)

Another developer wrote the first draft of the Fix-It back end shown below. The coordinator says it works, but it was never reviewed for security.

app_v1.py

```
 1  # app_v1.py: first draft of the Ward 42 Fix-It back end
 2  from flask import Flask, request, render_template_string
 3  import sqlite3, os
 4
 5  app = Flask(__name__)
 6  app.secret_key = "ward42"
 7
 8  @app.route("/search")
 9  def search():
10      street = request.args.get("street", "")
11      conn = sqlite3.connect("fixit.db")
12      rows = conn.execute(
13          f"SELECT id, category, status FROM reports WHERE street = '{street}'"
14      ).fetchall()
15      conn.close()
16      html = "<h2>Reports for " + street + "</h2><ul>"
17      for r in rows:
18          html += f"<li>#{r[0]} {r[1]}: {r[2]}</li>"
19      return render_template_string(html + "</ul>")
20
21  @app.route("/report/<int:report_id>/close", methods=["GET", "POST"])
22  def close_report(report_id):
23      conn = sqlite3.connect("fixit.db")
24      conn.execute("UPDATE reports SET status = 'Closed' WHERE id = ?",
25                   (report_id,))
26      conn.commit()
27      conn.close()
28      return "Report closed"
29
30  @app.route("/upload", methods=["POST"])
31  def upload():
32      photo = request.files["photo"]
33      photo.save(os.path.join("uploads", photo.filename))
34      return "Photo saved"
35
36  if __name__ == "__main__":
37      app.run(debug=True)
```

Question D1: Vulnerability analysis of app_v1.py (12 marks)

Identify at least **five** security vulnerabilities in `app_v1.py`. For each one, complete a table with these columns:

- **Line(s)** and the **name of the vulnerability** (for example SQL injection, XSS or CSRF);
- an **attack example** that shows how an attacker in Ward 42 could exploit it, and what personal information or function would be at risk;
- the **corrected code**, and an explanation of why the correction works.

Finally, explain in one paragraph which of your vulnerabilities would most seriously affect the association's obligations under POPIA, and why.

Question D2: Secure your own application (10 marks)

Apply security practices in your own Fix-It application, and provide evidence for each:

- server-side validation of every form field, in addition to your client-side validation;

- parameterised queries or an ORM for every database query;
- CSRF tokens on every form that changes data;
- secure session cookie settings (`Secure`, `HttpOnly` and `SameSite`) and at least two security headers, such as a Content Security Policy;
- a REST Client test file (`security_tests.http`) with at least four requests that attempt SQL injection, XSS, a missing CSRF token and an invalid upload, with screenshots that show your application rejecting each one.

PART E: DIGITAL MEDIA STORAGE, TRANSFER AND RETRIEVAL (18 MARKS)

Question E1: Secure photo upload and retrieval (10 marks)

- Allow residents to upload one photo per report. Accept only JPEG, PNG and WebP images, check both the file extension and the content type, and reject files larger than 2 MB with a clear message.
- Rename every uploaded file with `secure_filename` and a unique name, store it in a folder outside the templates, and save only the file path in the database (Table 1).
- Display a small thumbnail of the photo on the All reports page and the full photo on the report detail page, with suitable alt text.
- Explain why storing the photo as a file with its path in the database is better here than storing the image inside the database.

Question E2: JSON API, video and format choices (8 marks)

- Create a read-only API endpoint, `/api/reports/summary`, that returns the dashboard counts as **JSON**. Use JavaScript `fetch()` on the Dashboard page to retrieve the JSON and display it. Include a screenshot of the raw JSON response.
- Embed the tutorial video on the Home page with the HTML5 `<video>` element, provide an MP4 source and a WebM source, and add captions with a WebVTT `<track>` file. Use a short video of your own; it does not need to be professional.
- In a table, justify your choice of format for each type of media in your app (data, photos, video), comparing at least one alternative for each (for example JSON vs XML, JPEG vs PNG, MP4 vs streaming with HLS). Consider file size, browser support and accessibility.

PART F: REFLECTIVE QUESTION (10 MARKS)

Question F1: Reflecting on building Ward 42 Fix-It (10 marks)

Write a reflective account of 600 to 800 words about your own experience of building Ward 42 Fix-It. Use a recognised reflective model, such as Gibbs' reflective cycle, to structure your answer. In your reflection:

- describe one specific moment in the project that challenged you, for example a bug you could not find, a security flaw you discovered in your own code, or an accessibility problem you had overlooked;
- analyse why it happened and what it revealed about your knowledge, assumptions or working habits at the time;
- evaluate how your decisions affected the people who would use the app, such as an older resident with a screen reader, a resident with little mobile data, or the coordinator responsible for residents' personal information; and

- explain **two concrete changes** you will make to how you plan, build or test web applications in future, and why.

Write in the first person. You may refer to commits, screenshots or test results from your project as evidence. Part F is RED in the AI traffic light matrix: no AI tool may be used in any way.

PART G: DEPLOYMENT, DOCUMENTATION AND PROFESSIONAL PRESENTATION (10 MARKS)

Question G1: GitHub repository and README (5 marks)

- Keep your project in a GitHub repository with at least eight meaningful commits spread across the project, made from VS Code's Git tools
- Include a `README.md` that explains what the app does, how to install and run it, and the test coordinator login, and a `.gitignore` that excludes `venv`, the database file, the uploads folder and any secret keys.
- In your report, explain in a short paragraph which type of hosting from Section 8.4 of the Study Guide you would recommend to the association for the live app, and why GitHub Pages alone cannot host the Flask back end.

Question G2: Report presentation, referencing and AI declaration (5 marks)

Your report must have a cover page with your GitHub link, follow the order of the questions under clear headings, and include clear, labelled screenshots. Cite your sources in the text and in a reference list according to the STADIO Referencing Guide, and include the AI Usage Declaration required by the AI traffic light matrix.

MARKING RUBRIC

Each criterion is marked against the descriptors below. The mark range for each level appears in brackets. A criterion that is not attempted, or for which no evidence is submitted, earns **0** marks.

Part A: Web system architecture and HTTP analysis (15 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
A1 Architecture and request flow (8 marks)	**(7–8)** Diagram shows every required component with correctly labelled protocols and formats. The request flow for a photo submission is accurate and complete in all five steps. The SQL choice is justified with reference to Table 1.	**(5–6)** All components present, with minor labelling gaps. The flow is accurate but brief in one step. The justification is sound but generic.	**(4)** Some components or connections missing. The flow is partly correct. The justification is superficial.	**(1–3)** Diagram incomplete or incorrect. Flow and justification missing or wrong.
A2 HTTP analysis (7 marks)	**(6–7)** Clear captures of both requests. Method, URL, status, content types and two further headers identified correctly. The GET/POST difference and the need for HTTPS are explained accurately.	**(5)** Both requests captured, with one or two errors in the table or a brief explanation.	**(3–4)** One request analysed well, or both analysed with several errors.	**(1–2)** Screenshots without analysis, or the analysis is largely incorrect.

Part B: Responsive web application with a framework (30 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
B1 Semantic, accessible, responsive front end (10 marks)	**(8–10)** Shared base template with correct semantic structure and heading hierarchy. Every page works at all three widths; own media query used purposefully. POUR applied throughout: labels, alt text, contrast, relative sizes, keyboard use. Complete screenshots.	**(6–7)** Responsive and semantic, with minor accessibility gaps such as one unlabelled field or a skipped heading level.	**(4–5)** Works on desktop but layout breaks at one width, or accessibility is applied inconsistently. Some screenshots missing.	**(1–3)** Mostly non-semantic markup. Not responsive. Little attention to accessibility.
B2 Flask back end, database and dashboard (10 marks)	**(8–10)** Model stores every Table 1 field; references generated on the server. All routes work, including filtering and status updates. Coordinator pages protected; password hashed. Dashboard counts are correct.	**(6–7)** All features work, with a minor defect such as a filter that misses one case.	**(4–5)** Core create and list routes work; login, status updates or the dashboard are incomplete. Password stored insecurely.	**(1–3)** Back end largely does not work, or data is not stored in a database.

Criterion	Excellent	Good	Satisfactory	Needs improvement
B3 JavaScript interactivity (10 marks)	**(8–10)** Validation, live filter and draft all work smoothly with clear messages. Draft excludes personal information, with a reasoned explanation. External JS file with sensible error handling.	**(6–7)** All three features work, with minor usability issues or a thin explanation.	**(4–5)** Two of the three features work, or features work but personal data is stored in the draft.	**(1–3)** Little working JavaScript. Inline scripts only.

Part C: Mobile delivery as a Progressive Web App (15 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
C1 PWA conversion (8 marks)	**(7–8)** Complete, valid manifest. Service worker caches the app shell and serves a helpful offline page. Installation and offline behaviour are both shown clearly.	**(5–6)** Installs and works offline, with a minor manifest or caching gap.	**(4)** Manifest present but the service worker is incomplete, or offline behaviour is not demonstrated.	**(1–3)** PWA features attempted but do not work.
C2 Usability and efficiency across devices (7 marks)	**(6–7)** Two or more genuinely different devices tested and documented, with problems found and fixed. Lighthouse scores included. Three specific, well-reasoned data-saving changes.	**(5)** Two devices tested, with limited problem-solving or generic data-saving changes.	**(3–4)** Testing on one device only, or no Lighthouse audit.	**(1–2)** Little or no evidence of testing.

Part D: Security analysis and correction (22 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
D1 Vulnerability analysis of app_v1.py (12 marks)	**(10–12)** Five or more vulnerabilities correctly named, with line numbers. Realistic attack examples linked to the data at risk. Correct, working fixes with clear explanations. A well-argued POPIA paragraph.	**(8–9)** Five vulnerabilities identified, with one weak attack example or fix, or a brief POPIA paragraph.	**(6–7)** Three or four vulnerabilities identified, with partly correct fixes. POPIA link vague.	**(1–5)** One or two vulnerabilities identified, or fixes incorrect.
D2 Securing your own application (10 marks)	**(8–10)** All five practices implemented correctly. REST Client file contains four or more meaningful attack requests, each shown to be rejected.	**(6–7)** Four practices implemented well, or tests incomplete.	**(4–5)** Two or three practices implemented. Limited test evidence.	**(1–3)** Security practices largely absent.

Part E: Digital media storage, transfer and retrieval (18 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
E1 Secure photo upload and retrieval (10 marks)	**(8–10)** Upload enforces allowed types, content checks and the 2 MB limit with clear messages. Files renamed securely and stored outside templates; only the path is saved. Thumbnails and full images displayed with alt text. Sound justification.	**(6–7)** Upload and display work, with one missing check or a brief justification.	**(4–5)** Upload works but validation is weak, or images are not displayed correctly.	**(1–3)** Upload does not work, or files are stored unsafely.
E2 JSON API, video and format choices (8 marks)	**(7–8)** Endpoint returns correct JSON, retrieved and displayed with fetch(). Video plays with MP4 and WebM sources and working captions. Format table compares alternatives on size, support and accessibility.	**(5–6)** All three elements present, with minor gaps such as no WebM source or a thin comparison.	**(4)** Two elements present, or the format justification is descriptive rather than reasoned.	**(1–3)** One element present or largely incorrect.

Part F: Reflective question (10 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
F1a Description and critical analysis (6 marks)	**(5–6)** Specific, honest account of one real moment in the project, structured with a recognised model. Deep analysis of causes and of own assumptions, supported by evidence from the project.	**(4)** Clear account with some analysis of causes; evidence used occasionally.	**(3)** Mostly descriptive; analysis is general and could apply to any project.	**(1–2)** Vague or generic; no real analysis.
F1b Impact on users and future practice (4 marks)	**(4)** Thoughtful evaluation of the effect on specific users. Two concrete, justified changes to future practice. Within the word limit.	**(3)** Considers users and gives two changes, with limited justification.	**(2)** Users mentioned briefly; changes are vague, such as "test more".	**(1)** No consideration of users or of future practice.

Part G: Deployment, documentation and professional presentation (10 marks)

Criterion	Excellent	Good	Satisfactory	Needs improvement
G1 GitHub repository and README (5 marks)	**(5)** Eight or more meaningful commits across the project. Clear README and correct .gitignore. Sound hosting recommendation that explains the GitHub Pages limitation.	**(4)** Repository complete, with minor gaps in commits, README or reasoning.	**(3)** Few commits, or README incomplete, or secrets committed.	**(1–2)** Repository missing or almost empty.

Criterion	Excellent	Good	Satisfactory	Needs improvement
G2 Report, referencing and AI declaration (5 marks)	**(5)** Well-organised report with labelled screenshots. Sources cited in the text and in a reference list according to the STADIO Referencing Guide. Complete AI Usage Declaration.	**(4)** Minor organisation, language or referencing errors.	**(3)** Hard to follow; frequent referencing errors; declaration incomplete.	**(1–2)** Disorganised. No referencing and no declaration.

Mark summary

Part	Title	Available	Awarded
A	Web system architecture and HTTP analysis	15	
B	Responsive web application with a framework	30	
C	Mobile delivery as a Progressive Web App	15	
D	Security analysis and correction	22	
E	Digital media storage, transfer and retrieval	18	
F	Reflective question	10	
G	Deployment, documentation and professional presentation	10	
	TOTAL	**120**	

Marker's comments